Airline Seat Allocation with Multiple Nested Fare Classes

A study of Brumelle & McGill (1993)

Thomas Chrétienne Tom Cuel

ENSTA Paris, 3A, *Markov Decision Processes: Dynamic Programming and Applications* (M. Akian)

October 2026

1 The problem and what the article brings

An airline sells the seats of one cabin at several fares. Leisure travellers buy cheap tickets early, business travellers buy expensive ones late. Every cheap seat sold today may therefore be an expensive seat lost tomorrow, and the airline has to decide **how many seats to protect** for the expensive classes, without knowing how many of their customers will come.

Two answers existed before the article:

- **Two classes: Littlewood (1972).** Keep a seat for the expensive class as long as it earns more on average by waiting than by being sold now: $f_2 \leq f_1 \mathbb{P}[X_1 > p_1]$.
- n **classes: EMSRa, Belobaba (1987).** Apply Littlewood's rule to each pair of classes separately and add up the seats. Simple and used by airlines, but never proved optimal.

Brumelle & McGill (*Operations Research*, 1993), hereafter BMG, answer three questions:

(i) *What is optimal?* Fixed "protection levels", characterised by a simple condition.

(ii) *Is a fixed rule enough?* Yes, no rule using the booking history can do better.

(iii) *How wrong is EMSRa?* They show exactly where it fails, compare it numerically with the optimum, and give a case where it protects too many seats instead of too few.

In this report we explain these results in our own words, implement the method, reproduce the numerical part of the paper and add a sensitivity study (with the help of the Exercise 3.9 of the course notes).

2 The model

We keep the notation of the paper. Class 1 is the *most expensive*, $f_1 > f_2 > \cdots > f_n$, and X_k is the random demand of class k. Cheap classes book first: class n first, class 1 last. Demands are independent, there are no cancellations, and decisions depend only on the seats left.

A policy is a vector of **protection levels** $p = (p_1, \ldots, p_{n-1})$: p_k is the number of seats kept for the k most expensive classes. In practice, class $k+1$ is closed as soon as only p_k seats remain ($p_0 = 0$). With s seats left, class $k+1$ therefore sells $u = \min\left((s - p_k)^+, X_{k+1}\right)$ seats (what it is allowed, or its whole demand if smaller), and the remaining seats go on to the higher classes. This is the revenue recursion (9) of the paper:

$$R_{k+1}[s] = f_{k+1}\, u + R_k[s - u], \qquad R_1[s] = f_1 \min(s, X_1), \tag{1}$$

where $R_k[s]$ is the revenue of the k highest classes with s seats, and $ER_k[s]$ its expectation. The key quantity of the whole paper is the **marginal value of a seat**, $\delta ER_k[s] = ER_k[s] - ER_k[s-1]$: what one more seat brings, on average, to the k highest classes. (Because demand is an integer, the revenue moves in steps, so BMG work with left and right derivatives, for integer seats this is just the difference above.)

3 What the article proves

3.1 The optimal rule is a threshold

Theorem 1 (BMG). *If, for every k, the fare f_{k+1} lies between the left and right marginal values of ER_k at p_k ("condition (20)"), then each ER_k is concave in the number of seats and the policy p is optimal.*

So, protect a seat as long as it is worth more to the higher classes than the fare f_{k+1} we could get for it now: stop exactly where the two are equal. For two classes this is Littlewood's rule, so the paper generalises the known result.

Let's see how it works: we draw the expected revenue as a function of the seats s. It has three pieces: below p_k class $k+1$ sells nothing, so the slope is the value of a seat for the higher classes, while class $k+1$ sells, the slope is its fare f_{k+1}, once its demand is exhausted, we are back to the value for the higher classes. The curve stays concave (each new seat worth a bit less than the previous one) only if the slope keeps decreasing where the pieces meet, and this is exactly condition (20). Concavity then makes the revenue rise and fall only once as p_k grows, so (20) gives its top. A second result (Theorem 2) adds that with integer demand the optimal levels are integers: we never need to protect 72.4 seats.

3.2 No smarter rule can do better

Theorem 1 only compares fixed thresholds. One could imagine a cleverer rule that looks at how bookings went so far. BMG show that "sell one more seat or close the class?" is a *monotone optimal stopping* problem: the expected gain of selling one more seat changes sign only once, so the best possible behaviour is to stop at a fixed threshold. This is the strongest result of the paper.

3.3 Where EMSRa goes wrong

Under continuity assumptions, the optimal levels satisfy (eq. (7) of the paper)

$$f_{k+1} = f_1 \,\mathbb{P}\big[X_1 > p_1 \;\cap\; X_1 + X_2 > p_2 \;\cap \cdots \cap\; X_1 + \cdots + X_k > p_k\big]. \tag{2}$$

So, a protected seat earns the top fare only if *all* the remaining seats end up sold, so the condition involves one *joint* probability. EMSRa instead treats each higher class on its own, with *separate* probabilities $f_{k+1} = f_j \mathbb{P}[X_j > \pi_k^j]$, and adds the seats, $p_k = \sum_{j \le k} \pi_k^j$. It ignores the link between the events. Both methods agree on p_1 (only one class above), and differ from p_2 on.

3.4 Link with the course

Choosing the best p_k in (1) gives a Bellman equation, with ER_k^* the optimal expected revenue:

$$ER_0^* \equiv 0, \qquad ER_{k+1}^*[s] = \mathbb{E}\Big[\max_{(s-X_{k+1})^+ \le q \le s} \big\{f_{k+1}(s-q) + ER_k^*[q]\big\}\Big]. \tag{3}$$

The *stage* is the class being sold, the *state* the number of seats s, the *action* the number q of seats passed on to the higher classes, and the *reward* the fare times the seats sold. Computing $ER_1^*, ER_2^*, \ldots$ is value iteration on n stages, backwards in time since class 1 books last. Exercise 3.9 also shows that the marginal values are ordered, $\delta ER_k^* \le \delta ER_{k+1}^*$, which gives nested levels $p_1 \le p_2 \le \ldots$: more classes to protect, more seats kept.

4 Implementation

We follow Section 3.3 of BMG, in Python and with the paper's notation: compute ER_1^*, read p_1 where its marginal value crosses f_2, compute ER_2^* from (3), read p_2, and so on. The three main functions are:

- `solve_dp`: the exact solver: it returns ER_k^*, the marginal values and the levels p_k, in under a second for 200 seats
- `emsr_a`: the heuristic, coded as eqs. (2)–(5) of the paper
- `evaluate_nested_policy`: the *exact* expected revenue of any policy, obtained by following the probability of each number of seats left, class after class. There is no simulation noise, which matters since the gaps we measure are around 0.3 %.

Demands are Poisson, negative binomial, or normal rounded to integers (BMG's "discrete approximations to the normal").

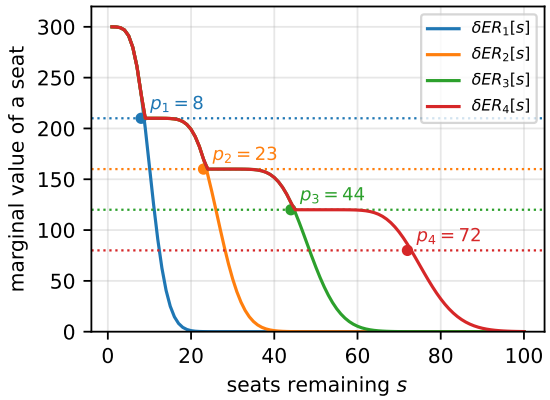
1. For two classes, the solver gives back Littlewood's rule and EMSRa must be optimal.
2. On the 5-class example of Section 5.1, all ER_k^* are concave, the levels are nested, and the exact evaluation of the optimal policy gives back the optimal value.

Policy iteration. We also solved the problem the other way round: start from some levels p, compute their exact revenue, improve every level greedily, $p_k \leftarrow \arg\max_q \{ER_k^p[q] - f_{k+1}q\}$, and repeat. On all our test cases it gives *exactly* the same levels and value as value iteration, so the two codes check each other. It needs only 2 to 4 rounds whatever n, but each round costs a full evaluation, so value iteration

stays faster. And if the levels are improved one after the other instead of all at once, a single round is enough: that round *is* value iteration. The two algorithms are the two ends of the same family.

5 Numerical results

5.1 What the solution looks like



Five classes, $f = (300, 210, 160, 120, 80)$, Poisson demands with means $(10, 15, 20, 25, 30)$, 100 seats. Each curve is the value of the next seat for the k highest classes. It decreases (concavity), and the protection level is read where it crosses the next fare: keep **8** seats for the top class, **23** for the two top classes, then **44** and **72**. The curves are stacked, hence nested levels.

Figure 1: Marginal value of a seat $\delta ER_k^*[s]$ (solid) and fares f_{k+1} (dotted). p_k is read at the crossing.

5.2 Reproduction of Table I of the paper

Setting of BMG Section 4: 100 seats, $f_1 = 1$, normal demands with means $40, 60, 80$ and standard deviations $16, 24, 32$, rounded to integers.

Ex.	f_3	f_2	p_1	p_2 EMSRa	p_2 optimal	EMSRa loss (%) ours	EMSRa loss (%) BMG	check of (2) $\mathbb{P}[X_1 > p_1]$	check of (2) $\mathbb{P}[X_1 > p_1, X_1 + X_2 > p_2]$
1	0.6	0.7	32	70	80	0.33	0.37	0.680	0.582
2	0.6	0.8	27	80	87	0.30	0.32	0.783	0.589
3	0.6	0.9	19	86	91	0.17	0.19	0.900	0.594
4	0.7	0.8	27	64	75	0.36	0.41	0.783	0.683
5	0.7	0.9	19	73	82	0.41	0.45	0.900	0.694
6	0.8	0.9	19	57	70	0.43	0.50	0.900	0.794

Table 1: BMG Table I recomputed. By (2), the last two columns should be close to f_2 and f_3.

- **All 18 protection levels are identical to the published ones.**
- Our revenue losses are slightly smaller (by 0.02 to 0.07 point): the paper does not say how it rounded the normal distribution.
- The last two columns are close to f_2 and f_3, but not equal (gap at most 0.02). This is not an error: with integer demand the probability moves by jumps and cannot hit the exact value. Eq. (2) is stated for continuous demand, condition (20) remains exact.

5.3 What moves the protection levels (our own study)

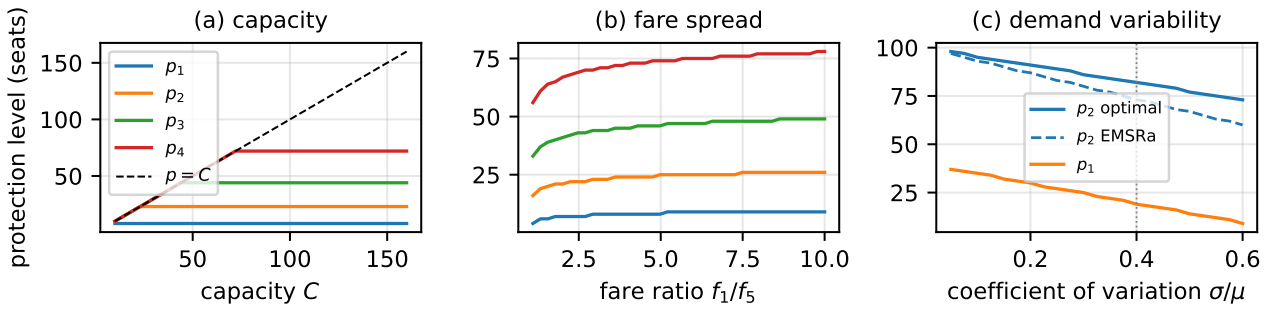


Figure 2: Optimal protection levels against (a) the capacity and (b) the ratio f_1/f_5 (5-class example, geometric fares), and (c) the coefficient of variation of demand (Table I setting with $f_2 = 0.9$, $f_3 = 0.7$, dashed: EMSRa).

The paper does not study sensitivity: Figure 2 shows the three effects we found.

(a) Capacity: no effect. The levels do not depend on the size of the cabin: they are only capped by it, $p_k(C) = \min(p_k^\infty, C)$. This surprised us at first, but it is logical: the value of a seat for the expensive classes depends on *their* demand, not on the total number of seats. A bigger cabin only leaves more seats to the cheap classes.

(b) Fare spread: a lot. The wider the fares, the more seats are protected, from $p = (4, 16, 33, 56)$ for nearly equal fares to $(9, 26, 49, 78)$ for a ratio of 10. More surprisingly, levels stay high even when fares are almost equal: keeping a seat that an expensive customer will buy anyway costs nothing.

(c) Demand uncertainty: less protection, worse EMSRa. With the same average demand, more uncertainty means fewer protected seats (p_2 goes from 98 to 73). Here the cheap fare is more than half the expensive one, so a seat is only worth protecting if it has more than one chance in two of being sold at the high fare: the level sits below the average demand, and more uncertainty pushes it further down. The gap between EMSRa and the optimum grows from 1 to 13 seats: when demand is almost known, separate and joint probabilities give almost the same answer.

5.4 EMSRa versus the optimum

Table II. With $f_2 = 0.9$ and $f_3 = 0.7$, the EMSRa loss falls as the cabin grows, as in the paper: 0.50, 0.41, 0.32, 0.22, 0.13 % for $C = 82, 100, \ldots, 160$, against 0.54, 0.45, 0.35, 0.24, 0.14 % published. With many seats, protecting well matters less.

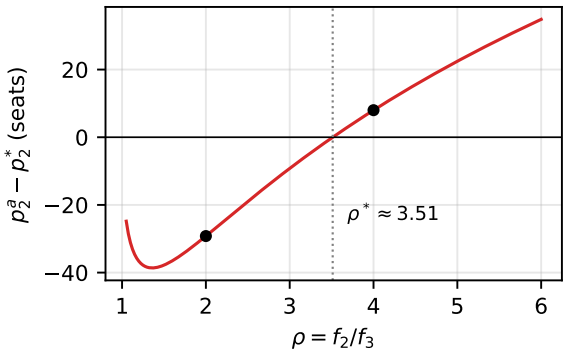


Figure 3: Section 4.1 of the paper: error of EMSRa on p_2 against $\rho = f_2/f_3$ (exponential demands, mean 100 seats). Dots: the paper's two cases.

EMSRa can protect too many seats. The paper shows this with exponential demands and mentions a threshold of "about 3.51" without detail. With exponential demands both rules can be solved by hand. Writing $p_2 = p_1 + \delta$, the optimum satisfies $(1 + \delta^*)e^{-\delta^*} = 1/\rho$, while EMSRa gives $\delta^a = 2 \ln \rho$. They agree when $\boldsymbol{\rho = 1 + 2 \ln \rho}$, i.e. $\rho^* \approx 3.513$: we recover the paper's value in closed form. Below it EMSRa protects too few seats (29 fewer for $\rho = 2$), above it too many (8 more for $\rho = 4$). Its error has no fixed sign, so it cannot be fixed by a simple safety margin.

6 Conclusion

Brumelle & McGill show that, because the expected revenue is concave in the number of seats, the seat allocation problem has a simple answer: a few fixed, nested protection levels, which no other policy can beat. Read as an MDP, their recursion is a value iteration over the fare classes: our code, checked by policy iteration, reproduces Table I exactly, the trend of Table II, and proves the 3.51 threshold. Two points surprised us: the levels do not depend on capacity, and EMSRa loses less than 0.5 % of revenue while being wrong by 5 to 13 seats, because the revenue is flat around the optimum (near p_k one seat more or less is worth about the fare). The limits come from the model itself: independent demands, a strict cheap-before-expensive order, and no cancellations.

A Going further (beyond the article)

EMSRb. A later heuristic of Belobaba groups all higher classes into one, with an average fare. On ten test cases (2 to 8 classes) its mean loss is 0.02%, against 0.23% for EMSRa.

Arrival order. With the model of Lee & Hersh (1993), where requests can arrive in any order, the same demand ($f = (300, 150, 80)$, means $(10, 15, 20)$, 30 seats) brings +5.7% if arrivals are mixed and +8.2% if expensive customers come first. Cheap-before-expensive is the worst case for the airline, which is exactly why protection levels are needed.

No control. First come, first served loses 2.7% in our 5-class example, and up to 48% when the cabin is half the average demand.

Monte Carlo simulation. To check our exact revenue computation with an independent method, we also simulated many flights of our 5-class example: we draw the demand of each class at random, apply the optimal protection levels, and average the revenues. The simulated average matches the exact value within 0.04 %, and we found the same agreement for all the policies and scenarios we tested, which confirms our computation.

B Core of the solver

The code uses the paper's notation: `fares[k-1]` $= f_k$, `p[k]` $= p_k$ ($p_0 = 0$), `ER[k, s]` $= ER_k^*[s]$. The full code is provided with the report.

```python
def solve_dp(fares, demand_pmfs, C): # fares f_1 > ... > f_n (class 1 books last)
    n = len(fares), ss = np.arange(C + 1)
    ER = np.zeros((n + 1, C + 1)) # ER[0] = ER_0 = 0
    p = np.zeros(n, dtype=int) # p[0] = p_0 = 0
    for k in range(n): # ER_k -> ER_{k+1}: class k+1 is selling
        f, pmf = fares[k], demand_pmfs[k] # f_{k+1}, pmf of X_{k+1}
        g = -f * ss + ER[k] # g(q) = ER_k[q] - f_{k+1} q
        p[k] = int(np.argmax(g)) # p_k = smallest argmax <=> condition (20)
        for s in range(C + 1):
            S = np.maximum.accumulate(g[s::-1])[::-1] # S[j] = max g[j..s]
            total = sum(pmf[x] * S[s - x] for x in range(min(s - 1, len(pmf) - 1) + 1))
            total += pmf[s:].sum() * S[0] if s < len(pmf) else 0.0 # X_{k+1} >= s: q in [0, s]
            ER[k + 1, s] = f * s + total
    return {"ER": ER, "p": p, "value": ER[n, C]} # (excerpt: marginal values omitted)
```